

Chapter 15 — Monotonic Queue / Deque

0. What You Will Learn in This Chapter

Monotonic Queue / Deque is an advanced queue pattern used when a problem needs the **minimum or maximum inside a moving window**.

It is closely related to Monotonic Stack, but instead of solving nearest greater/smaller boundary problems, it solves **sliding window optimization** problems.

A monotonic queue is usually implemented with a deque.

It supports:

- push from back
- pop from back
- pop from front
- read best value from front

By the end of this chapter, you should understand:

- What a deque is
- What a monotonic queue is
- Why normal queue is not enough
- Why heap is sometimes less ideal
- How to maintain maximum in a sliding window
- How to maintain minimum in a sliding window
- How to store indices instead of values
- How to remove expired elements
- How to solve fixed-window maximum/minimum problems
- How to solve prefix-sum + deque problems
- How to solve DP optimization with deque
- How to recognize when monotonic deque is needed
- How to avoid common off-by-one mistakes

Core problems covered in this chapter:

1. Sliding Window Maximum
 2. Sliding Window Minimum
 3. Shortest Subarray With Sum at Least K
 4. Constrained Subsequence Sum
 5. Jump Game VI
 6. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
 7. Maximum Robots Within Budget
 8. Continuous Subarrays
 9. Minimum Cost to Make Array Equalindromic, concept overlap
 10. Max Value of Equation
 11. Count Non-Decreasing Subarrays After K Operations, advanced
 12. Sliding Window Median, comparison with ordered structures
-

1. What Is a Deque?

A deque means:

double-ended queue

It allows insertion and removal from both ends.

In C++:

```
deque<int> dq;
```

Common operations:

```
dq.push_back(x);
```

```
dq.pop_back();
```

```
dq.push_front(x);
```

```
dq.pop_front();
```

```
dq.front();
```

```
dq.back();
```

```
dq.empty();
```

```
dq.size();
```

A normal queue only lets you push at the back and pop from the front.

A deque is more flexible.

2. What Is a Monotonic Queue?

A monotonic queue is a deque that maintains elements in a monotonic order.

Two common types:

2.1 Monotonic Decreasing Queue

Values decrease from front to back.

```
front -> 9, 7, 5, 2 <- back
```

The maximum is always at the front.

Used for:

- sliding window maximum
- maximum DP transition
- best previous value in a range

2.2 Monotonic Increasing Queue

Values increase from front to back.

```
front -> 1, 3, 4, 8 <- back
```

The minimum is always at the front.

Used for:

- sliding window minimum
- shortest subarray with prefix sums
- minimum DP transition

3. Why Monotonic Queue Exists

Suppose you need the maximum of every window of size k.

Example:

```
nums = [1,3,-1,-3,5,3,6,7]
```

```
k = 3
```

Windows:

```
[1,3,-1] -> 3
```

```
[3,-1,-3] -> 3
```

```
[-1,-3,5] -> 5
```

```
[-3,5,3] -> 5
```

```
[5,3,6] -> 6
```

```
[3,6,7] -> 7
```

Brute force checks each window:

```
for each window:
```

```
    scan k elements
```

Time:

```
O(n * k)
```

A monotonic deque does this in $O(n)$:

- Keep useful candidates only.
- Remove elements outside the window.
- Keep largest element at the front.

Each element is pushed once and popped at most once.

4. Core Need / Why Use Monotonic Queue Over Other Patterns?

4.1 Why Not Normal Queue?

A normal queue can tell you the oldest element.

It cannot tell you the maximum or minimum efficiently.

Sliding window max needs:

```
maximum of current window
```

not just front element.

4.2 Why Not Heap?

A heap can get max/min, but removing expired elements is harder.

You can use heap with lazy deletion, but it becomes:

```
O(n log n)
```

Monotonic deque gives:

```
O(n)
```

for many window max/min problems.

4.3 Why Not Monotonic Stack?

Monotonic stack is for nearest greater/smaller boundaries.

Monotonic queue is for a moving window where elements expire from the front.

The key difference:

Monotonic Stack	next/previous greater/smaller
Monotonic Queue	max/min in sliding window

4.4 Why Not Sorting Each Window?

Sorting each window is too slow.

Even maintaining a balanced tree is usually:

$O(n \log k)$

Deque can do $O(n)$ when only max/min is needed.

5. When to Use Monotonic Queue

Use monotonic queue when the problem asks for:

5.1 Sliding Window Maximum

Clues:

- maximum in every window of size k
- fixed-size window max
- moving maximum

Example:

- Sliding Window Maximum

5.2 Sliding Window Minimum

Clues:

- minimum in every window
- shortest/cheapest value in recent range

Example:

- Sliding Window Minimum
- shortest subarray with prefix constraints

5.3 Max-Min Window Constraint

Clues:

- longest subarray where $\max - \min \leq \text{limit}$
- maintain both max and min in current window

Examples:

- Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
- Continuous Subarrays

5.4 Prefix Sum Optimization

Clues:

- shortest subarray with sum at least K
- negative numbers exist
- sliding window fails
- prefix sums need increasing deque

Example:

- Shortest Subarray With Sum at Least K

5.5 DP With Window Maximum

Clues:

- $dp[i]$ depends on max of previous k dp values
- jump within k positions
- constrained subsequence

Examples:

- Constrained Subsequence Sum
 - Jump Game VI
-

6. When NOT to Use Monotonic Queue

6.1 Need Median

Use heaps or multiset.

Example:

- Sliding Window Median

A deque cannot maintain median.

6.2 Need Arbitrary Order Statistics

Use ordered set, Fenwick tree, segment tree, or heaps.

6.3 Need All Values Sorted

Use multiset or sorting.

6.4 Window Does Not Move Linearly

Monotonic queue works when left and right move forward.

If window changes arbitrarily, deque may not work.

6.5 Need Nearest Greater/Smaller Boundary

Use monotonic stack.

7. Core Intuition

A monotonic deque removes values that can never be useful again.

For sliding window maximum:

If current value is greater than a previous value, then that previous value can never be maximum in any future window that includes current.

So remove it.

Example:

`nums = [1, 3]`

When 3 arrives, 1 is useless.

Any future window containing 1 and 3 will prefer 3.

So pop 1.

8. Mental Models

8.1 Candidate List

Deque stores only candidates for best value.

For maximum:

- values are decreasing
- front is best candidate
- back contains weaker newer candidates

8.2 Expired Elements

Window moves right.

When left boundary passes an index, that index is expired.

Remove it from front if it is there.

8.3 Stronger New Element Deletes Weaker Old Element

For max deque:

Current larger value deletes smaller values from back.

For min deque:

Current smaller value deletes larger values from back.

8.4 Front Is the Answer

Once deque is maintained:

```
nums[dq.front()]
```

is current window maximum or minimum.

9. C++ Tools Needed

9.1 Include Deque

```
#include <deque>
```

9.2 Basic Deque Usage

```
deque<int> dq;
```

```
dq.push_back(i);
```

```
dq.pop_back();
```

```
dq.pop_front();
```

```
dq.front();
```

```
dq.back();
```

```
dq.empty();
```

9.3 Store Indices, Not Values

Usually store indices:

```
deque<int> dq;
```

Why?

Because you need to know if an element expired:

```
dq.front() <= i - k
```

If you store values only, you cannot tell whether the value is outside the window.

10. Core Templates

Template 1 — Sliding Window Maximum

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> answer;

    for (int i = 0; i < nums.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        while (!dq.empty() && nums[dq.back()] <= nums[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
            answer.push_back(nums[dq.front()]);
        }
    }

    return answer;
}
```

Template 2 — Sliding Window Minimum

```
vector<int> minSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> answer;

    for (int i = 0; i < nums.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        while (!dq.empty() && nums[dq.back()] >= nums[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
```

```

        answer.push_back(nums[dq.front()]);
    }
}

return answer;
}

```

Template 3 — Longest Window With Max-Min $\leq$ Limit

Use two deques:

- max deque decreasing
- min deque increasing

```

int longestSubarray(vector<int>& nums, int limit) {
    deque<int> maxDq;
    deque<int> minDq;

    int left = 0;
    int best = 0;

    for (int right = 0; right < nums.size(); right++) {
        while (!maxDq.empty() && nums[maxDq.back()] < nums[right]) {
            maxDq.pop_back();
        }

        while (!minDq.empty() && nums[minDq.back()] > nums[right]) {
            minDq.pop_back();
        }

        maxDq.push_back(right);
        minDq.push_back(right);

        while (nums[maxDq.front()] - nums[minDq.front()] > limit) {
            if (maxDq.front() == left) {
                maxDq.pop_front();
            }

            if (minDq.front() == left) {
                minDq.pop_front();
            }

            left++;
        }
    }
}

```

```

        best = max(best, right - left + 1);
    }

    return best;
}

```

Template 4 — Prefix Sum + Increasing Deque

Used for shortest subarray with sum at least K.

```

int shortestSubarray(vector<int>& nums, int k) {
    int n = nums.size();

    vector<long long> prefix(n + 1, 0);

    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + nums[i];
    }

    deque<int> dq;
    int answer = n + 1;

    for (int i = 0; i <= n; i++) {
        while (!dq.empty() && prefix[i] - prefix[dq.front()] >= k) {
            answer = min(answer, i - dq.front());
            dq.pop_front();
        }

        while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {
            dq.pop_back();
        }

        dq.push_back(i);
    }

    return answer == n + 1 ? -1 : answer;
}

```

Template 5 — DP With Window Maximum

```

for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() < i - k) {
        dq.pop_front();
    }
}

```

```

dp[i] = nums[i] + max(0, dp[dq.front()]);

while (!dq.empty() && dp[dq.back()] <= dp[i]) {
    dq.pop_back();
}

dq.push_back(i);
}

```

11. Major Variations of Monotonic Queue

Fixed window maximum	decreasing deque	Sliding Window Maximum
Fixed window minimum	increasing deque	Sliding Window Minimum
Max-min constraint	two deques	Longest Continuous Subarray
Prefix sum shortest subarray	increasing prefix deque	Shortest Subarray $\geq K$
DP max transition	decreasing dp deque	Jump Game VI
DP constrained subsequence	decreasing dp deque	Constrained Subsequence Sum
Equation max	decreasing transformed value	Max Value of Equation
Variable budget window	max deque + sum	Maximum Robots

12. Full Problem Walkthrough 1 — LeetCode 239: Sliding Window Maximum

12.1 Problem Summary

Given array nums and integer k, return maximum value in every contiguous window of size k.

Example:

```
nums = [1,3,-1,-3,5,3,6,7]
```

```
k = 3
```

Output:

```
[3,3,5,5,6,7]
```

12.2 How to Identify This Pattern

Problem asks:

- sliding window
- fixed size k
- maximum in each window

Normal sliding window cannot get max in $O(1)$.

Use monotonic decreasing deque.

12.3 Brute Force Idea

For each window, scan all k elements and find max.

Time:

```
O(n * k)
```

Too slow.

12.4 Optimized Intuition

Maintain a deque of indices whose values are decreasing.

Rules:

13. Remove expired indices from front.
 14. Remove smaller/equal values from back.
 15. Push current index.
 16. Front is max.
-

12.5 C++ Code

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> answer;

        for (int i = 0; i < nums.size(); i++) {
            while (!dq.empty() && dq.front() <= i - k) {
                dq.pop_front();
            }
        }
    }
};
```

```

while (!dq.empty() && nums[dq.back()] <= nums[i]) {
    dq.pop_back();
}

dq.push_back(i);

if (i >= k - 1) {
    answer.push_back(nums[dq.front()]);
}
}

return answer;
}
};

```

12.6 Dry Run

Input:

nums = [1,3,-1,-3,5,3,6,7]

k = 3

0	1	1	
1	3	3	
2	-1	3, -1	3
3	-3	3, -1, -3	3
4	5	5	5
5	3	5, 3	5
6	6	6	6
7	7	7	7

Answer:

[3,3,5,5,6,7]

12.7 Correctness Explanation

The deque always stores indices inside the current window in decreasing value order. Any value smaller than the current value and behind it can never become maximum later, so it is removed. Therefore, the front always holds the maximum candidate.

12.8 Derived Complexity Proof

Each index is pushed once.

Each index is popped from front or back at most once.

Total deque operations:

$O(n)$

Time:

$O(n)$

Space:

$O(k)$

12.9 Edge Cases

- $k = 1$
- $k = \text{nums.size}()$
- all increasing values
- all decreasing values
- duplicate values
- negative values

12.10 Common Mistakes

Mistake 1 — Storing values instead of indices

Need indices to know expiration.

Mistake 2 — Not removing expired elements

Wrong if old max leaves the window.

Mistake 3 — Using $<$ vs $<=$ carelessly

Both can work, but using $<=$ removes older duplicates and keeps deque smaller.

12.11 Similar Problems

- Sliding Window Minimum
- Longest Continuous Subarray
- Jump Game VI
- Constrained Subsequence Sum

13. Full Problem Walkthrough 2 — Sliding Window Minimum

13.1 Problem Summary

Given array `nums` and window size `k`, return minimum of every window.

Example:

`nums = [1,3,-1,-3,5,3,6,7]`

`k = 3`

Output:

[-1,-3,-3,-3,3,3]

13.2 How to Identify This Pattern

Problem asks for minimum in every fixed window.

Use monotonic increasing deque.

13.3 C++ Code

```
vector<int> minSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> answer;

    for (int i = 0; i < nums.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        while (!dq.empty() && nums[dq.back()] >= nums[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
            answer.push_back(nums[dq.front()]);
        }
    }

    return answer;
}
```

13.4 Complexity

Time:

$O(n)$

Space:

$O(k)$

13.5 Key Difference From Maximum

For maximum:

$\text{nums}[\text{dq.back()}] \leq \text{nums}[i]$

For minimum:

```
nums[dq.back()] >= nums[i]
```

14. Full Problem Walkthrough 3 — LeetCode 862: Shortest Subarray With Sum at Least K

14.1 Problem Summary

Given integer array `nums` and integer `k`, return length of shortest non-empty subarray with sum at least `k`.

If none exists, return `-1`.

Example:

```
nums = [2,-1,2]
```

```
k = 3
```

Output:

```
3
```

14.2 How to Identify This Pattern

Problem asks:

- shortest subarray
- sum at least `K`
- negative numbers may exist

Normal sliding window fails with negative numbers.

Use prefix sums and monotonic increasing deque.

14.3 Brute Force Idea

Check every subarray sum.

Time:

```
O(n2)
```

14.4 Optimized Intuition

Let:

```
prefix[i] = sum of nums[0..i-1]
```

Subarray sum from `j` to `i - 1` is:

```
prefix[i] - prefix[j]
```

Need:

```
prefix[i] - prefix[j] >= k
```

For each `i`, we want earliest useful `j`, but also shortest length.

Deque keeps prefix indices with increasing prefix values.

14.5 C++ Code

```
class Solution {
public:
    int shortestSubarray(vector<int>& nums, int k) {
        int n = nums.size();

        vector<long long> prefix(n + 1, 0);

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }

        deque<int> dq;
        int answer = n + 1;

        for (int i = 0; i <= n; i++) {
            while (!dq.empty() && prefix[i] - prefix[dq.front()] >= k) {
                answer = min(answer, i - dq.front());
                dq.pop_front();
            }

            while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {
                dq.pop_back();
            }

            dq.push_back(i);
        }

        return answer == n + 1 ? -1 : answer;
    }
};
```

14.6 Why Pop From Front?

If:

```
prefix[i] - prefix[dq.front()] >= k
```

then subarray from `dq.front()` to `i - 1` is valid.

We update answer.

Then we pop it because for future `i`, using this old start would only make length longer.

14.7 Why Pop From Back?

If current prefix is smaller or equal than a previous prefix, then previous prefix is worse.

Suppose:

$\text{prefix}[i] \leq \text{prefix}[j]$

and $i > j$.

For any future end, using i gives:

- equal or larger subarray sum
- shorter length

So j is useless.

14.8 Complexity

Time:

$O(n)$

Space:

$O(n)$

14.9 Edge Cases

- negative numbers
 - no valid subarray
 - one element satisfies
 - large sums requiring long long
 - k very large
-

14.10 Similar Problems

- Minimum Size Subarray Sum, positive-only sliding window
 - Constrained Subsequence Sum
 - Prefix sum + monotonic deque problems
-

15. Full Problem Walkthrough 4 — LeetCode 1425: Constrained Subsequence Sum

15.1 Problem Summary

Given array `nums` and integer `k`, return maximum sum of a non-empty subsequence such that adjacent chosen indices differ by at most `k`.

Example:

`nums = [10,2,-10,5,20]`

`k = 2`

Output:

37

Choose:

$10 + 2 + 5 + 20$

15.2 How to Identify This Pattern

This is DP.

Let:

$dp[i]$ = best subsequence sum ending at i

Then:

$dp[i] = \text{nums}[i] + \max(0, dp[j])$

where:

$i - k \leq j < i$

Need maximum $dp[j]$ in sliding window of previous k indices.

Use monotonic decreasing deque.

15.3 C++ Code

```
class Solution {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> dp(n);
        deque<int> dq;

        int answer = nums[0];

        for (int i = 0; i < n; i++) {
            while (!dq.empty() && dq.front() < i - k) {
                dq.pop_front();
            }

            dp[i] = nums[i];

            if (!dq.empty()) {
                dp[i] += max(0, dp[dq.front()]);
            }

            while (!dq.empty() && dp[dq.back()] <= dp[i]) {
                dq.pop_back();
            }
        }
    }
};
```

```

    }

    dq.push_back(i);

    answer = max(answer, dp[i]);
}

return answer;
}
};

```

15.4 Dry Run Concept

For each index:

- choose current number alone
- or extend best previous subsequence within distance k

Deque front stores best previous dp.

15.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

Can optimize DP array if careful, but $O(n)$ is clear.

15.6 Edge Cases

- all negative numbers
- $k = 1$
- $k \geq n$
- large positive and negative values

16. Full Problem Walkthrough 5 — LeetCode 1696: Jump Game VI

16.1 Problem Summary

Given array `nums` and integer `k`.

You start at index 0.

From index `i`, you can jump to any index:

`i + 1` to `i + k`

Return maximum score to reach last index.

Score is sum of visited values.

Example:

```
nums = [1,-1,-2,4,-7,3]
k = 2
```

Output:

7

16.2 How to Identify This Pattern

This is DP:

```
dp[i] = nums[i] + max(dp[j])
```

where:

```
j is in [i-k, i-1]
```

Need max in sliding window.

Use monotonic decreasing deque.

16.3 C++ Code

```
class Solution {
public:
    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> dp(n);
        deque<int> dq;

        dp[0] = nums[0];
        dq.push_back(0);

        for (int i = 1; i < n; i++) {
            while (!dq.empty() && dq.front() < i - k) {
                dq.pop_front();
            }

            dp[i] = nums[i] + dp[dq.front()];

            while (!dq.empty() && dp[dq.back()] <= dp[i]) {
                dq.pop_back();
            }

            dq.push_back(i);
        }

        return dp[n - 1];
    }
};
```

```
}  
};
```

16.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

16.5 Similar Problems

- Constrained Subsequence Sum
- Sliding Window Maximum
- Minimum Cost problems with bounded jumps

17. Full Problem Walkthrough 6 — LeetCode 1438: Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

17.1 Problem Summary

Given array `nums` and integer `limit`, return length of longest contiguous subarray where absolute difference between any two elements is at most `limit`.

This means:

$$\max(\text{window}) - \min(\text{window}) \leq \text{limit}$$

Example:

`nums = [8,2,4,7]`
`limit = 4`

Output:

2

17.2 How to Identify This Pattern

Need longest sliding window with:

$$\max - \min \leq \text{limit}$$

Need maintain `max` and `min` dynamically.

Use two monotonic deques.

17.3 C++ Code

```

class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        deque<int> maxDq;
        deque<int> minDq;

        int left = 0;
        int best = 0;

        for (int right = 0; right < nums.size(); right++) {
            while (!maxDq.empty() && nums[maxDq.back()] < nums[right]) {
                maxDq.pop_back();
            }

            while (!minDq.empty() && nums[minDq.back()] > nums[right]) {
                minDq.pop_back();
            }

            maxDq.push_back(right);
            minDq.push_back(right);

            while (nums[maxDq.front()] - nums[minDq.front()] > limit) {
                if (maxDq.front() == left) {
                    maxDq.pop_front();
                }

                if (minDq.front() == left) {
                    minDq.pop_front();
                }

                left++;
            }

            best = max(best, right - left + 1);
        }

        return best;
    }
};

```

17.4 Dry Run

nums = [8,2,4,7]

limit = 4

- [8,2]: max-min = 6, invalid
- shrink to [2]
- [2,4]: valid length 2
- [2,4,7]: max-min = 5, invalid
- shrink to [4,7], valid length 2

Answer:

2

17.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

17.6 Alternatives

A multiset can solve in:

$O(n \log n)$

Deque is faster:

$O(n)$

18. Full Problem Walkthrough 7 — LeetCode 2398: Maximum Number of Robots Within Budget

18.1 Problem Summary

You have robots with:

chargeTimes[i]

runningCosts[i]

Cost of running a window of robots from l to r:

$\max(\text{chargeTimes}[l..r]) + \text{windowSize} * \sum(\text{runningCosts}[l..r])$

Return maximum number of consecutive robots within budget.

18.2 How to Identify This Pattern

Need longest window with:

- max charge time
- sum running costs
- budget constraint

Use:

- monotonic decreasing deque for max charge
 - running sum for running costs
 - sliding window shrink
-

18.3 C++ Code

```
class Solution {
public:
    int maximumRobots(vector<int>& chargeTimes,
                     vector<int>& runningCosts,
                     long long budget) {
        deque<int> maxDq;

        long long sumRunning = 0;
        int left = 0;
        int best = 0;

        for (int right = 0; right < chargeTimes.size(); right++) {
            sumRunning += runningCosts[right];

            while (!maxDq.empty() && chargeTimes[maxDq.back()] <=
chargeTimes[right]) {
                maxDq.pop_back();
            }

            maxDq.push_back(right);

            while (!maxDq.empty()) {
                long long windowSize = right - left + 1;
                long long cost = chargeTimes[maxDq.front()] + windowSize *
sumRunning;

                if (cost <= budget) {
                    break;
                }

                if (maxDq.front() == left) {
                    maxDq.pop_front();
                }
            }
        }
    }
};
```

```

        sumRunning -= runningCosts[left];
        left++;
    }

    best = max(best, right - left + 1);
}

return best;
}
};

```

18.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

18.5 Common Mistakes

- Recomputing max each time
- Forgetting long long
- Shrinking without updating running sum
- Not removing expired max index

19. Full Problem Walkthrough 8 — LeetCode 1499: Max Value of Equation

19.1 Problem Summary

Given sorted points by x-coordinate.
Find max value of:

$$y_i + y_j + |x_i - x_j|$$

where:

$$i < j$$

$$x_j - x_i \leq k$$

Since points are sorted and $i < j$:

$$|x_i - x_j| = x_j - x_i$$

Expression becomes:

$$(y_i - x_i) + (y_j + x_j)$$

For each j , need maximum:

$y_j - x_i$

among valid previous points.

19.2 How to Identify Monotonic Queue

For every current point, need best previous transformed value within x -distance k .

This is sliding window maximum over transformed values.

Use decreasing deque.

19.3 C++ Code

```
class Solution {
public:
    int findMaxValueOfEquation(vector<vector<int>>& points, int k) {
        deque<int> dq;
        int answer = INT_MIN;

        for (int j = 0; j < points.size(); j++) {
            int xj = points[j][0];
            int yj = points[j][1];

            while (!dq.empty() && xj - points[dq.front()][0] > k) {
                dq.pop_front();
            }

            if (!dq.empty()) {
                int i = dq.front();
                answer = max(answer, yj + xj + points[i][1] - points[i][0]);
            }

            int currentValue = yj - xj;

            while (!dq.empty()) {
                int back = dq.back();
                int backValue = points[back][1] - points[back][0];

                if (backValue >= currentValue) {
                    break;
                }
            }

            dq.pop_back();
        }
    }
};
```

```
        dq.push_back(j);
    }

    return answer;
}
};
```

19.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

20. Full Problem Walkthrough 9 — LeetCode 2762: Continuous Subarrays

20.1 Problem Summary

Return number of continuous subarrays where absolute difference between any two elements is at most 2.

Equivalent condition:

$\max(\text{window}) - \min(\text{window}) \leq 2$

Example:

nums = [5,4,2,4]

Output:

8

20.2 How to Identify This Pattern

Need count subarrays satisfying max-min condition.

Use two deques for max and min.

For each right, after shrinking to valid window, all subarrays ending at right and starting from left..right are valid.

Add:

$\text{right} - \text{left} + 1$

20.3 C++ Code

```

class Solution {
public:
    long long continuousSubarrays(vector<int>& nums) {
        deque<int> maxDq;
        deque<int> minDq;

        int left = 0;
        long long answer = 0;

        for (int right = 0; right < nums.size(); right++) {
            while (!maxDq.empty() && nums[maxDq.back()] < nums[right]) {
                maxDq.pop_back();
            }

            while (!minDq.empty() && nums[minDq.back()] > nums[right]) {
                minDq.pop_back();
            }

            maxDq.push_back(right);
            minDq.push_back(right);

            while (nums[maxDq.front()] - nums[minDq.front()] > 2) {
                if (maxDq.front() == left) {
                    maxDq.pop_front();
                }

                if (minDq.front() == left) {
                    minDq.pop_front();
                }

                left++;
            }

            answer += right - left + 1;
        }

        return answer;
    }
};

```

20.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

21. Full Problem Walkthrough 10 — LeetCode 2944: Minimum Number of Coins for Fruits

21.1 Problem Summary

You are given prices of fruits.

When you buy fruit i , you can get some next fruits for free according to the problem rule.

Return minimum coins needed to get all fruits.

This problem can be solved using DP and a monotonic deque optimization.

21.2 How to Identify This Pattern

The DP has the form:

$dp[i] = price[i] + \min(dp[j])$

where j lies in a future valid range.

Whenever DP needs min or max over a moving range, monotonic deque may apply.

21.3 Conceptual Template

For DP minimum over range:

while expired:

pop front

$dp[i] = cost[i] + dp[dq.front()]$

while $dp[new] \leq dp[dq.back()]$:

pop back

push new

21.4 Key Takeaway

Not every monotonic deque problem is a direct sliding window max.

Sometimes it appears inside DP as:

best value among recent/future states

22. Full Problem Walkthrough 11 — Sliding Window Median Comparison

22.1 Problem Summary

Given array and window size k , return median of every window.

Example:

```
nums = [1,3,-1,-3,5,3,6,7]
```

```
k = 3
```

Output:

```
[1,-1,-1,3,5,6]
```

22.2 Why Monotonic Queue Is Not Enough

Median is not just maximum or minimum.

A deque can maintain:

- max
- min

But median requires middle order statistic.

Need:

- two heaps with lazy deletion
- multiset
- ordered statistic tree

22.3 Lesson

Use monotonic deque only when the window query is about:

- max
- min
- best transformed value
- monotonic DP transition

Do not force it for median.

23. Mini Problem 1 — Fixed Window Maximum Manual Function

```
vector<int> windowMax(vector<int>& nums, int k) {  
    deque<int> dq;  
    vector<int> result;  
  
    for (int i = 0; i < nums.size(); i++) {  
        if (!dq.empty() && dq.front() <= i - k) {  
            dq.pop_front();  
        }  
    }  
}
```

```

while (!dq.empty() && nums[dq.back()] <= nums[i]) {
    dq.pop_back();
}

dq.push_back(i);

if (i >= k - 1) {
    result.push_back(nums[dq.front()]);
}
}

return result;
}

```

24. Mini Problem 2 — Fixed Window Minimum Manual Function

```

vector<int> windowMin(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> result;

    for (int i = 0; i < nums.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        while (!dq.empty() && nums[dq.back()] >= nums[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
            result.push_back(nums[dq.front()]);
        }
    }

    return result;
}

```

25. Mini Problem 3 — Maximum of $dp[i-k..i-1]$

```
deque<int> dq;

for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() < i - k) {
        dq.pop_front();
    }

    if (!dq.empty()) {
        dp[i] = nums[i] + dp[dq.front()];
    }

    while (!dq.empty() && dp[dq.back()] <= dp[i]) {
        dq.pop_back();
    }

    dq.push_back(i);
}
```

26. Mini Problem 4 — Minimum of Prefix Values

Used when you want best previous prefix.

```
while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {
    dq.pop_back();
}

dq.push_back(i);
```

The deque front is the smallest useful prefix.

27. Common Edge Cases for Monotonic Queue

27.1 $k = 1$

Each window contains one element.

Maximum/minimum is the element itself.

27.2 $k = n$

Only one window.

27.3 Duplicates

Using $\leq$ or $<$ can both work in many max problems, but affects which duplicate index is kept.

Commonly use:

```
nums[dq.back()] <= nums[i]
```

to remove older duplicate values.

27.4 Expired Indices

Always remove indices outside window.

For window ending at i of size k , valid indices are:

```
i - k + 1 to i
```

Expired condition:

```
dq.front() <= i - k
```

27.5 Negative Numbers

For sum problems with negatives, use prefix + deque, not normal sliding window.

27.6 Large Sums

Use long long.

28. Common Mistakes and Wrong Code

Mistake 1 — Storing Values Instead of Indices

Wrong for expiration:

```
deque<int> dq; // values
```

Correct:

```
deque<int> dq; // indices
```

Mistake 2 — Forgetting Expiration

If old index leaves window but remains in deque, answer becomes wrong.

Mistake 3 — Maintaining Wrong Direction

For maximum, deque should be decreasing.

For minimum, deque should be increasing.

Mistake 4 — Updating Answer Too Early

For fixed window size k , only output when:

```
i >= k - 1
```

Mistake 5 — Thinking Nested While Is $O(n^2)$

Each index enters and leaves deque at most once.

Total $O(n)$.

Mistake 6 — Using Monotonic Queue for Median

Deque gives max/min, not median.

29. Pattern Recognition Checklist

Ask:

17. Do I need max/min in a moving window?
18. Does the window move left to right?
19. Do old elements expire?
20. Is heap possible but $O(n \log n)$?
21. Can weaker previous elements be removed permanently?
22. Is the query only max/min, not median?
23. Does DP need max/min over last k states?
24. Does prefix sum need best previous prefix?
25. Do I need both max and min in current window?
26. Can each index be pushed and popped once?

If yes, monotonic queue is likely useful.

30. Interview Explanation Templates

30.1 Sliding Window Maximum

I use a deque of indices, maintaining values in decreasing order. The front is always the maximum of the current window. Before adding a new index, I remove expired indices from the front and remove smaller values from the back because they can never become maximum while the current value is present.

30.2 Sliding Window Minimum

This is symmetric to maximum. I maintain an increasing deque, so the front always stores the minimum of the current window.

30.3 Max-Min Constraint Window

I maintain two deques: one decreasing for maximum and one increasing for minimum. The current window is valid when max minus min is at most the limit. If invalid, I move the left pointer and remove expired indices.

30.4 Prefix Sum + Deque

For shortest subarray with sum at least K , I use prefix sums. The deque stores candidate start indices with increasing prefix sums. Smaller prefix sums are better starts, and once a start gives a valid subarray, I pop it to seek a shorter one.

30.5 DP Window Maximum

If $dp[i]$ depends on the maximum DP value among the last k indices, I maintain those candidate indices in a decreasing deque. This makes each transition $O(1)$ amortized.

31. Complexity Patterns

Sliding Window Maximum	$O(n)$	$O(k)$
Sliding Window Minimum	$O(n)$	$O(k)$
Max-Min Window Constraint	$O(n)$	$O(n)$
Shortest Subarray $\geq K$	$O(n)$	$O(n)$
Constrained Subsequence Sum	$O(n)$	$O(n)$
Jump Game VI	$O(n)$	$O(n)$
Maximum Robots	$O(n)$	$O(n)$
Max Value of Equation	$O(n)$	$O(n)$
Continuous Subarrays	$O(n)$	$O(n)$
Sliding Window Median	Not deque-only	$O(n \log k)$ with other structures

32. Time-Boxed Practice Guidance

Intro Monotonic Queue

Target:

20–30 minutes

Problems:

- Sliding Window Maximum
- Sliding Window Minimum

Core Medium Problems

Target:

30–45 minutes

Problems:

- Longest Continuous Subarray With Absolute Diff $\leq$ Limit
- Jump Game VI
- Constrained Subsequence Sum

Hard Monotonic Queue Problems

Target:

45–75 minutes

Problems:

- Shortest Subarray With Sum at Least K
 - Maximum Robots Within Budget
 - Max Value of Equation
 - Continuous Subarrays
-

33. Expanded Practice Set

33.1 Must-Do Medium

27. Sliding Window Maximum
28. Sliding Window Minimum
29. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
30. Jump Game VI
31. Constrained Subsequence Sum
32. Continuous Subarrays
33. Maximum Number of Robots Within Budget

33.2 Strong Interview Practice

34. Shortest Subarray With Sum at Least K
35. Max Value of Equation
36. Minimum Number of Coins for Fruits
37. Minimum Cost With Bounded Jumps
38. Count Non-Decreasing Subarrays After K Operations
39. Sliding Window Median, compare with heaps/multiset

33.3 Hard / Stretch

40. Shortest Subarray With Sum at Least K
 41. Maximum Number of Robots Within Budget
 42. Max Value of Equation
 43. Count Non-Decreasing Subarrays After K Operations
 44. Sliding Window Median
 45. Constrained Subsequence Sum
 46. Jump Game VI
-

34. Suggested Study Order

Step 1 — Fixed Window Max/Min

Start with:

47. Sliding Window Maximum
48. Sliding Window Minimum

Goal:

Understand deque maintenance and expired indices.

Step 2 — Variable Window With Max-Min

Solve:

49. Longest Continuous Subarray With Absolute Diff $\leq$ Limit
50. Continuous Subarrays

Goal:
Learn two-deque window constraints.

Step 3 — DP Optimization

Solve:
51. Jump Game VI
52. Constrained Subsequence Sum

Goal:
Recognize max DP transition over last k.

Step 4 — Prefix Sum + Deque

Solve:
53. Shortest Subarray With Sum at Least K

Goal:
Understand increasing prefix deque and negative numbers.

Step 5 — Advanced Transformations

Solve:
54. Max Value of Equation
55. Maximum Robots Within Budget

Goal:
Learn transformed values and budget windows.

35. Pattern Decision Table

max in every window	decreasing deque	Sliding Window Maximum
min in every window	increasing deque	Sliding Window Minimum
max-min $\leq$ limit	two deques	Longest Continuous Subarray
count valid max-min windows	two deques + counting	Continuous Subarrays
shortest sum $\geq K$ with negatives	prefix + increasing deque	LC 862
dp[i] needs max previous k	decreasing deque over dp	Jump Game VI
best previous transformed value	decreasing deque	Max Value of Equation
max charge in budget	decreasing deque +	Maximum Robots

Problem Clue	Deque Strategy	Example
window	sum	
median in window	not monotonic deque	Sliding Window Median

36. Final Cheat Sheet

Max Deque

```

while (!dq.empty() && dq.front() <= i - k) {
    dq.pop_front();
}

while (!dq.empty() && nums[dq.back()] <= nums[i]) {
    dq.pop_back();
}

dq.push_back(i);

maxValue = nums[dq.front()];

```

Min Deque

```

while (!dq.empty() && dq.front() <= i - k) {
    dq.pop_front();
}

while (!dq.empty() && nums[dq.back()] >= nums[i]) {
    dq.pop_back();
}

dq.push_back(i);

minValue = nums[dq.front()];

```

Two Deques for Max-Min

```

while (nums[maxDq.front()] - nums[minDq.front()] > limit) {
    if (maxDq.front() == left) {
        maxDq.pop_front();
    }

    if (minDq.front() == left) {
        minDq.pop_front();
    }
}

```

```
    left++;  
}
```

Prefix + Deque

```
while (!dq.empty() && prefix[i] - prefix[dq.front()] >= k) {  
    answer = min(answer, i - dq.front());  
    dq.pop_front();  
}  
  
while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {  
    dq.pop_back();  
}  
  
dq.push_back(i);
```

DP Max Deque

```
while (!dq.empty() && dq.front() < i - k) {  
    dq.pop_front();  
}  
  
dp[i] = nums[i] + dp[dq.front()];  
  
while (!dq.empty() && dp[dq.back()] <= dp[i]) {  
    dq.pop_back();  
}  
  
dq.push_back(i);
```

37. What to Master Before Chapter 16

Before moving to Linked List, make sure you can:

- Use C++ deque
- Explain why deque stores indices
- Maintain decreasing deque for maximum
- Maintain increasing deque for minimum
- Remove expired indices
- Solve Sliding Window Maximum
- Solve Sliding Window Minimum
- Use two deques for max-min window constraints
- Solve Longest Continuous Subarray
- Solve Shortest Subarray With Sum at Least K

- Solve Jump Game VI
 - Solve Constrained Subsequence Sum
 - Explain why monotonic deque is $O(n)$
 - Know when heap/multiset is needed instead
-

38. Core Takeaway

Monotonic Queue / Deque is used when a window moves and you need the best value inside it.

The key idea is:

Keep only useful candidates.

For maximum:

remove smaller values from the back
front is maximum

For minimum:

remove larger values from the back
front is minimum

The deque supports both:

- removing expired old elements from the front
- removing weaker new candidates from the back

That is why it is perfect for sliding windows.

Each index is pushed once and popped at most once, so the total time is $O(n)$.

Monotonic Queue is the bridge between Sliding Window, Monotonic Stack, Prefix Sum, and DP optimization.
